



Advanced Certification Programme in Agentic and Generative AI

Capstone Projects List

Implementation Guidelines for All Projects

Phase 1: Foundation

- Define the project scope, user personas, business objective, architecture block diagram, and success metrics.
- Set up the development environment using Google Colab, local Python, VS Code, or GitHub Codespaces.
- Prepare datasets, clean raw data, validate schema, and create synthetic samples where needed.
- Set up knowledge bases using vector databases such as ChromaDB or FAISS.
- Configure API keys securely using environment variables or .env files.

Phase 2: Core Build

- Develop data ingestion pipelines for CSV, PDF, JSON, APIs, and database sources.
- Build RAG pipelines using LangChain or LlamaIndex with ChromaDB/FAISS retrievers.
- Design agentic workflows using LangGraph, CrewAI, AutoGen, or MCP-based tool integrations.
- Implement structured outputs using Pydantic models or JSON parsers.
- Build backend APIs using FastAPI for model inference, data retrieval, and workflow execution.

Phase 3: Integration and Automation

- Connect external tools and services using APIs, webhooks, or MCP servers.
- Integrate dashboards or interfaces using Streamlit, Gradio, or React-based frontends.
- Add workflow automation using FastAPI endpoints, scheduled jobs, or event-driven triggers.
- Store processed outputs in SQLite, PostgreSQL, Airtable, or Pandas DataFrames.
- Add logging, error handling, retries, and fallback responses for robust execution.

Phase 4: Testing and Refinement

- Unit-test individual pipeline components, tools, and agent nodes.
- Evaluate RAG output quality using RAGAS, retrieval accuracy, faithfulness, and answer relevance.

- Test prompts with multiple input scenarios and refine using few-shot examples.
- Validate structured outputs, dashboard updates, and API responses.
- Conduct error analysis and improve prompts, retrieval logic, thresholds, and fallback handling.

Phase 5: Deployment and Documentation

- Deploy the application using Streamlit Cloud, HuggingFace Spaces, Docker, or cloud platforms.
- Containerize the solution using Docker for reproducibility and easier deployment.
- Document the architecture, data flow, agent workflow, APIs, and database schema.
- Prepare an evaluation report with metrics, screenshots, sample outputs, and limitations.
- Record a 5-minute walkthrough demo showing the full end-to-end workflow.

Common Tools Stack

- **Programming & Development:** Python, Google Colab, VS Code, GitHub Codespaces
- **Orchestration & Agents:** LangChain, LangGraph, CrewAI, AutoGen, MCP
- **LLMs:** Groq models, OpenAI models, Llama 3, Mistral, DeepSeek, Gemma, Phi, HuggingFace Transformers
- **Embeddings:** Sentence Transformers, all-MiniLM, all-mpnet, BGE embeddings, domain-specific embeddings, OpenAI embeddings
- **Vector Databases:** ChromaDB, FAISS, Qdrant, Pinecone, Supabase
- **RAG & Evaluation:** LangChain LCEL, RAGAS, BERTScore, METEOR, scikit-learn metrics
- **Backend APIs:** FastAPI, Flask, Uvicorn, Pydantic
- **UI / Interface:** Streamlit, Gradio, React, HuggingFace Spaces
- **Databases:** SQLite, PostgreSQL, Airtable, Pandas DataFrames
- **Deployment:** Docker, HuggingFace Spaces, Streamlit Cloud, GitHub Codespaces
- **Monitoring & Logging:** Python logging, LangSmith, Langfuse, OpenTelemetry
- **Data Processing:** Pandas, NumPy, PyMuPDF, BeautifulSoup, Requests, SQLAlchemy

Capstone Project 1

Title: Multi-Agent Code Review & Auto-Debugging System

Overview and Problem Statement:

This project aims to develop an AI-powered multi-agent code review and auto-debugging system that automates software code analysis, vulnerability detection, debugging, patch generation, and test case creation using Large Language Models (LLMs), LangGraph-based orchestration, Retrieval-Augmented Generation (RAG), and static code analysis tools.

Software development teams spend significant time performing manual code reviews, identifying bugs, validating coding standards, and fixing vulnerabilities. Manual reviews often depend heavily on reviewer expertise, leading to inconsistent quality, missed security issues, delayed development cycles, and repetitive debugging efforts.

The proposed system will build a LangGraph-orchestrated multi-agent pipeline where specialized AI agents collaboratively analyze source code for security vulnerabilities, logical bugs, coding standard violations, and performance issues. The platform will also generate structured patches, automated test cases, and grounded recommendations using RAG over OWASP and CWE security references.

The project aims to improve software quality, accelerate debugging workflows, reduce security risks, and enhance developer productivity through intelligent AI-assisted code review automation.

Dataset:

The project will utilize a combination of publicly available datasets, benchmark datasets, and synthetic code samples:

1. Code Vulnerability Dataset:
 - a. Source: CodeXGLUE and OWASP benchmark datasets
 - b. Content: Vulnerable and secure source code examples, CWE classifications, security issue labels, and vulnerability descriptions
[CodeXGLUE Dataset](#)
2. Bug Detection Dataset:
 - a. Source: Defects4J and CodeSearchNet datasets
 - b. Content: Real-world buggy code samples, bug-fix pairs, stack traces, and corrected implementations
[Defects4J Dataset](#)
[CodeSearchNet Dataset](#)



3. Code Quality and Style Dataset:
 - a. Source: Open-source GitHub repositories
 - b. Content: Python codebases with PEP8 violations, complexity issues, code smells, and performance-related problems
4. Security Knowledge Base:
 - a. Source: OWASP Top 10 and CWE references
 - b. Content: Security vulnerability descriptions, mitigation strategies, attack patterns, and remediation guidelines
[OWASP Top 10](#)

Methodology:

1. Data Preparation:
 - a. Collect vulnerable, buggy, and high-quality code samples
 - b. Preprocess source code and extract AST (Abstract Syntax Tree) representations
 - c. Organize security and debugging knowledge base documents
2. Multi-Agent Pipeline Development:
 - a. Build LangGraph-based orchestration workflow
 - b. Design specialized AI agents for different review tasks
 - c. Route code snippets dynamically between agents
3. Security Analysis Agent:
 - a. Detect OWASP Top-10 vulnerabilities
 - b. Identify insecure coding patterns
 - c. Retrieve grounded security references using RAG
4. Bug Detection Agent:
 - a. Analyze logical and runtime issues
 - b. Detect syntax errors, faulty logic, and exception-prone code
 - c. Suggest debugging recommendations
5. Style and Performance Agent:
 - a. Validate PEP8 compliance
 - b. Measure code complexity using Radon
 - c. Analyze performance bottlenecks and code smells
6. Patch Generation Agent:
 - a. Generate structured code fixes and diffs using LLMs and Pydantic schemas
 - b. Recommend optimized implementations
7. Test Generation Agent:
 - a. Automatically generate pytest-based unit test cases
 - b. Validate patched code against generated tests
8. RAG Integration:



- a. Store OWASP, CWE, and coding guideline documents in Vectorstore like ChromaDB
 - b. Retrieve contextual security and debugging references during analysis
9. Evaluation:
 - a. Measure vulnerability detection accuracy
 - b. Evaluate bug detection precision and false positives
 - c. Test patch quality and unit test effectiveness

Challenges:

1. Multi-Agent Coordination: Synchronize multiple specialized agents efficiently within the orchestration pipeline
2. Accurate Vulnerability Detection: Detect subtle security vulnerabilities and reduce false positives
3. Logical Bug Identification: Identify runtime and logic-related issues beyond syntax validation
4. Hallucination in Code Generation: Prevent incorrect or unsafe patches generated by LLMs
5. AST and Code Parsing Complexity: Correctly parse and analyze large and complex codebases
6. Test Case Reliability: Generate meaningful and executable unit tests for diverse code scenarios
7. Scalability: Handle large repositories and multiple concurrent code review requests efficiently

References:

1. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation <https://github.com/microsoft/CodeXGLUE>
2. Defects4J Bug Dataset <https://github.com/rjust/defects4j>
3. OWASP Top 10 Security Risks <https://owasp.org/www-project-top-ten/>
4. MITRE Common Weakness Enumeration (CWE) <https://cwe.mitre.org/>
5. Radon Python Code Metrics Tool <https://radon.readthedocs.io/en/latest/>
6. Pylint Documentation <https://pylint.pycqa.org/en/latest/>
7. Pytest Documentation <https://docs.pytest.org/en/stable/>
8. CodeBERT: A Pre-Trained Model for Programming and Natural Languages <https://arxiv.org/abs/2002.08155>

Capstone Project 2

Title: Autonomous Research Paper Summarization & Knowledge Graph Builder

Overview and Problem Statement:

This project aims to develop an AI-powered autonomous research paper summarization and knowledge graph generation system that can automatically ingest research papers, extract key information, build interconnected knowledge graphs, and support intelligent question-answering over scientific literature using Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), LangGraph orchestration, and graph-based reasoning.

Researchers and academicians publish thousands of papers every week across domains such as Artificial Intelligence, Machine Learning, Healthcare, and Data Science. Keeping up with the rapidly growing literature requires significant manual effort in reading, summarizing, highlighting, and connecting ideas across papers. Important relationships between methods, datasets, findings, metrics, and concepts are often missed due to information overload.

The proposed system will automate the end-to-end research analysis workflow by fetching papers from arXiv, extracting structured entities and relationships, generating concise summaries, building interactive knowledge graphs, and enabling semantic Q&A over connected research concepts. The platform will help researchers discover hidden relationships, accelerate literature reviews, and improve research exploration efficiency.

Dataset:

The project will utilize a combination of publicly available research datasets and real-time paper sources:

1. arXiv Research Papers:
 - a. Source: arXiv API
 - b. Content: Research papers from domains such as cs.AI, cs.LG, stat.ML, NLP, and Computer Vision
[arXiv API Documentation](#)
2. SciREX Dataset:
 - a. Source: Hugging Face Datasets
 - b. Content: Scientific entity extraction, relation extraction, and structured research annotations
[SciREX Dataset](#)
3. Semantic Scholar Open Research Corpus:
 - a. Source: Semantic Scholar



- b. Content: Citation networks, paper metadata, authors, venues, abstracts, and references

[Semantic Scholar Open Research Corpus](#)

- 4. Synthetic Research Papers:
 - a. Source: LLM-generated synthetic papers
 - b. Content: Simulated research papers for stress testing extraction and graph generation pipelines

Methodology:

- 1. Data Ingestion Agent:
 - a. Fetch research papers using arXiv API
 - b. Extract paper text using PyMuPDF
 - c. Clean and preprocess research documents
- 2. Document Chunking and Embedding:
 - a. Split papers into overlapping text chunks using LangChain RecursiveCharacterTextSplitter
 - b. Generate embeddings using sentence-transformers
 - c. Store embeddings in Vectorestore like ChromaDB for semantic retrieval
- 3. Knowledge Extraction Agent:
 - a. Use LLMs to extract entities such as methods, datasets, metrics, concepts, and findings
 - b. Generate structured Pydantic outputs for entity-relation extraction
 - c. Deduplicate entities using fuzzy matching and embedding similarity
- 4. Knowledge Graph Construction:
 - a. Build graph nodes and edges using NetworkX DiGraph
 - b. Store relationships between papers, methods, datasets, and concepts
 - c. Enrich graph with metadata such as citation count, publication year, and venues
- 5. Summarization Pipeline:
 - a. Generate concise research paper summaries
 - b. Produce section-wise and overall paper summaries
 - c. Highlight key findings and contributions
- 6. Q&A Agent with RAG:
 - a. Perform graph traversal for entity-centric queries
 - b. Combine graph context with ChromaDB semantic search
 - c. Generate grounded answers using RAG pipelines
- 7. Visualization Interface:
 - a. Visualize knowledge graph using PyVis
 - b. Build interactive interface using Gradio

- c. Display related papers, connected concepts, and Q&A responses
- 8. Evaluation:
 - a. Evaluate entity extraction accuracy
 - b. Measure graph connectivity and retrieval relevance
 - c. Assess summary quality and Q&A performance

Challenges:

1. Scientific Text Complexity: Handle highly technical terminology and domain-specific language
2. Entity and Relation Extraction: Accurately identify relationships between concepts, datasets, methods, and findings
3. Graph Scalability: Efficiently manage large-scale research knowledge graphs
4. Duplicate Entity Resolution: Merge similar entities with different naming conventions
5. Hallucination in Summarization: Prevent incorrect or unsupported research summaries generated by LLMs
6. Semantic Retrieval Accuracy: Improve relevance of retrieved paper chunks during RAG-based Q&A
7. Multi-Agent Coordination: Synchronize ingestion, extraction, summarization, and graph-building agents

References:

1. LangGraph Documentation <https://langchain-ai.github.io/langgraph/>
2. LangChain Documentation <https://python.langchain.com/docs/introduction/>
3. Chroma Vector Database Documentation <https://docs.trychroma.com/>
4. NetworkX Documentation <https://networkx.org/documentation/stable/>
5. PyVis Documentation <https://pyvis.readthedocs.io/en/latest/>
6. PyMuPDF Documentation <https://pymupdf.readthedocs.io/en/latest/>
7. Sentence Transformers Documentation <https://www.sbert.net/>
8. Gradio Documentation <https://www.gradio.app/docs>



Capstone Project 3

Title: Agentic Clinical Decision Support System with Medical RAG

Overview and Problem Statement:

This project aims to develop an AI-powered clinical decision support system that assists healthcare professionals in retrieving medical knowledge, checking drug interactions, analyzing clinical queries, and generating evidence-grounded responses using Retrieval-Augmented Generation (RAG), LangGraph-based multi-agent orchestration, and biomedical Large Language Models (LLMs).

Doctors and clinicians work with enormous volumes of medical information including clinical guidelines, treatment protocols, drug interactions, and continuously evolving research findings. During busy clinical workflows, manually searching through lengthy medical documents can delay decision-making and increase the risk of missing critical information. Incorrect dosage recommendations, overlooked drug interactions, or outdated treatment knowledge may lead to serious patient safety concerns.

The proposed system will build a multi-agent clinical reasoning pipeline capable of classifying medical queries, retrieving trusted medical evidence, checking drug contraindications, enforcing safety guardrails, and generating citation-grounded medical responses. The platform aims to support clinicians with faster access to reliable medical knowledge while reducing hallucinations and improving decision support workflows.

Dataset:

The project will utilize publicly available biomedical datasets, medical literature, and healthcare knowledge sources:

1. PubMed Abstracts:
 - a. Source: NCBI Entrez API
 - b. Content: Biomedical abstracts, clinical studies, treatment research, and medical findings
[PubMed API](#)
2. CDC and WHO Clinical Guidelines:
 - a. Source: CDC and WHO public medical guideline repositories
 - b. Content: Clinical protocols, disease guidelines, prevention strategies, and treatment procedures
[CDC Clinical Guidelines](#)
[WHO Guidelines Repository](#)
3. OpenFDA Drug Labels Dataset:



- a. Source: OpenFDA API
 - b. Content: Drug labels, dosage information, contraindications, warnings, and drug interaction data
[OpenFDA API](#)
4. Medical QA Benchmark Datasets:
- a. Source: Hugging Face Medical Datasets
 - b. Content: Clinical reasoning questions, USMLE-style QA, medical MCQs, and diagnosis-related queries
[MedQA Dataset](#)

Methodology:

1. Medical Knowledge Base Preparation:
 - a. Collect biomedical abstracts, clinical guidelines, and drug label information
 - b. Extract and preprocess medical documents
 - c. Chunk and embed medical text using PubMedBERT embeddings
2. RAG Pipeline Development:
 - a. Store embeddings in FAISS and ChromaDB
 - b. Implement semantic retrieval and BM25 keyword search
 - c. Retrieve grounded medical evidence for clinical queries
3. Multi-Agent Clinical Reasoning Pipeline:
 - a. Build LangGraph-based orchestration workflow
 - b. Develop specialized agents for triage, retrieval, safety validation, synthesis, and drug interaction analysis
4. Triage Agent:
 - a. Classify medical queries into categories such as dosage, symptoms, drug interaction, treatment protocol, and clinical guidance
5. Retrieval Agent:
 - a. Perform hybrid retrieval using semantic search and keyword search
 - b. Retrieve evidence from PubMed abstracts and clinical guidelines
6. Drug Interaction Agent:
 - a. Query OpenFDA APIs and local drug interaction databases
 - b. Detect contraindications and potential medication conflicts
7. Synthesis Agent:
 - a. Generate citation-grounded medical responses using biomedical LLMs
 - b. Restrict answers strictly to retrieved evidence to minimize hallucinations
8. Safety and Guardrails Agent:
 - a. Apply rule-based safety checks
 - b. Reject unsafe or high-risk requests such as direct prescription generation or diagnosis confirmation

- c. Add clinical disclaimers where necessary
- 9. Human-in-the-Loop Workflow:
 - a. Route high-severity or critical queries for clinician review before final response generation
- 10. Evaluation and Interface:
 - a. Evaluate system performance on MedQA and MedMCQA benchmarks
 - b. Build interactive Gradio-based clinical assistant interface with evidence citations

Challenges:

1. Medical Hallucination Prevention: Ensure generated medical responses remain grounded in trusted evidence
2. Clinical Safety and Ethics: Prevent unsafe recommendations and inappropriate medical advice
3. Drug Interaction Accuracy: Detect complex contraindications and medication conflicts reliably
4. Biomedical Retrieval Relevance: Improve retrieval quality across large-scale medical corpora
5. Complex Medical Terminology: Handle highly technical biomedical vocabulary and abbreviations
6. Multi-Agent Coordination: Synchronize reasoning across triage, retrieval, safety, and synthesis agents
7. Regulatory and Deployment Risks: Address ethical concerns and limitations of AI-assisted clinical decision systems

References:

1. PubMed Entrez API Documentation <https://www.ncbi.nlm.nih.gov/books/NBK25501/>
2. PubMedBERT Model Documentation <https://huggingface.co/microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract>



Capstone Project 4

Title: Agentic Cybersecurity Threat Intelligence & Incident Response Advisor

Overview and Problem Statement:

This project aims to develop an AI-powered multi-agent cybersecurity threat intelligence and incident response platform using LangGraph, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and threat intelligence knowledge bases. The system will automate security log analysis, threat correlation, attack mapping, vulnerability enrichment, and incident response playbook generation to help cybersecurity teams respond faster and more accurately to security incidents.

Organizations face thousands of cybersecurity alerts daily from firewalls, SIEM tools, intrusion detection systems, endpoint security platforms, and network monitoring solutions. Security analysts must manually investigate logs, identify Indicators of Compromise (IoCs), correlate attack patterns, verify vulnerabilities, and generate incident response procedures. This process is time-consuming, error-prone, and difficult to scale due to the large volume of alerts and fragmented threat intelligence sources.

The proposed system will use AI agents to parse security logs, extract IoCs, enrich threat intelligence using external APIs and knowledge bases, map attack behavior to MITRE ATT&CK techniques, identify kill-chain stages, and generate contextual incident response playbooks using LLMs and RAG pipelines.

Dataset:

The project will utilize a combination of publicly available cybersecurity datasets, threat intelligence repositories, and synthetic security logs.

1. MITRE ATT&CK STIX Dataset
 - a. Source: MITRE ATT&CK Framework
 - b. Content: ATT&CK tactics, techniques, procedures (TTPs), threat actor behaviors, mitigations, and detection guidance
[MITRE ATT&CK STIX Data](#)
2. NVD CVE JSON Feeds
 - a. Source: National Vulnerability Database (NVD)
 - b. Content: Vulnerability records, CVE descriptions, CVSS scores, affected systems, and severity information
[NVD CVE JSON Feeds](#)
3. CISA Known Exploited Vulnerabilities (KEV) Catalogue



- a. Source: Cybersecurity and Infrastructure Security Agency (CISA)
 - b. Content: Actively exploited vulnerabilities and remediation guidance
[CISA Known Exploited Vulnerabilities Catalogue](#)
4. CTI-Bench Dataset
 - a. Source: Hugging Face
 - b. Content: Cyber Threat Intelligence extraction and evaluation benchmark dataset
[CTI-Bench Dataset](#)
5. Synthetic Security Log Dataset
 - a. Source: Generated using Python and LLMs
 - b. Content: Simulated Apache, Syslog, Windows Event Logs, phishing alerts, malware activity, suspicious IP activity, and APT attack simulations

Methodology:

1. Environment Setup
 - a. Configure development environment using Python, LangChain, LangGraph, ChromaDB, and Hugging Face models
 - b. Install cybersecurity libraries such as stix2, taxii2-client, and mitreattack-python
 - c. Setup vector database for threat intelligence retrieval
2. Threat Intelligence Knowledge Base
 - a. Parse and embed MITRE ATT&CK techniques into ChromaDB using cybersecurity embedding models
 - b. Ingest CVE records and CISA KEV catalogue into vector database
 - c. Build semantic retrieval pipelines for ATT&CK and vulnerability correlation
3. Log Analysis Agent
 - a. Parse Apache, Nginx, Syslog, and Windows Event Logs
 - b. Extract Indicators of Compromise (IoCs) such as IP addresses, domains, URLs, and file hashes
 - c. Enrich IoCs using AbuseIPDB and VirusTotal APIs
4. Threat Correlation Agent
 - a. Cluster related IoCs and suspicious activities using embedding-based similarity and clustering techniques
 - b. Map observed attack patterns to MITRE ATT&CK techniques and tactics
 - c. Identify kill-chain stages and attack progression
5. Incident Response Playbook Agent
 - a. Retrieve relevant incident response procedures from NIST SP 800-61 knowledge base
 - b. Use LLMs to generate contextual step-by-step incident response playbooks



- c. Generate remediation recommendations and investigation workflows
- 6. Dashboard and Alerting System
 - a. Build Gradio or Streamlit dashboard for visualization
 - b. Display IoC timelines, ATT&CK heatmaps, CVE severity distributions, and incident summaries
 - c. Implement human-in-the-loop workflows for critical severity incidents

Challenges:

1. Large-Scale Log Processing: Efficiently process and analyze high-volume security logs in real time
2. Threat Correlation Accuracy: Correctly map IoCs and attack behavior to ATT&CK techniques and kill-chain stages
3. False Positive Reduction: Minimize noise and false alerts generated during threat detection
4. Threat Intelligence Integration: Combine multiple threat intelligence sources into a unified retrieval pipeline
5. Contextual Incident Response Generation: Generate relevant and environment-specific incident response procedures
6. Scalability: Support concurrent incident analysis and large-scale threat intelligence retrieval
7. Human-in-the-loop Coordination: Balance AI automation with analyst review and validation
8. Security and Privacy: Safeguard sensitive log and incident information during analysis workflows

References:

1. ChromaDB Documentation <https://docs.trychroma.com/>
2. VirusTotal API Documentation <https://docs.virustotal.com/reference/overview>
3. AbuseIPDB API Documentation <https://docs.abuseipdb.com/>
4. NIST SP 800-61 Incident Response Guide
<https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final>
5. Lewis, Patrick, et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." <https://arxiv.org/abs/2005.11401>

Capstone Project 5

Title: Multi-Modal Document Intelligence Agent for Invoice & Form Processing

Overview and Problem Statement:

This project aims to develop an AI-powered multi-modal document intelligence system for automated invoice, receipt, and form processing using vision-language models (VLMs), OCR pipelines, LangGraph-based multi-agent orchestration, Retrieval-Augmented Validation, and structured data extraction workflows.

Organizations process hundreds of invoices, receipts, purchase orders, and forms from different vendors in varying layouts and formats. Most of these documents are handled manually, where operators read documents, identify key fields, validate entries, and input the information into enterprise systems. This process is time-consuming, error-prone, and difficult to scale.

Traditional OCR systems can extract plain text but often struggle with Complex layouts, Multi-page documents, Poor scan quality, Handwritten fields, Inconsistent vendor formats. These limitations lead to incorrect data extraction, delayed processing, payment mismatches, and compliance issues.

The proposed system will automate the end-to-end document processing pipeline by combining OCR, vision-language models, validation agents, semantic matching, and auto-correction workflows. The platform will extract structured invoice data, validate extracted information, correct low-confidence fields, and maintain an auditable database of processed records.

Dataset:

The project will utilize publicly available document understanding datasets and synthetic invoice datasets:

1. SROIE Dataset:
 - a. Source: Hugging Face Datasets
 - b. Content: Scanned receipts with key-value annotations including invoice number, totals, dates, and vendor names
[SROIE Dataset](#)
2. RVL-CDIP Dataset:
 - a. Source: Hugging Face Datasets



- b. Content: Document images across multiple categories including invoices, forms, letters, and receipts
[RVL-CDIP Dataset](#)
- 3. CORD Dataset:
 - a. Source: Clova AI Receipt Dataset
 - b. Content: Multi-lingual receipt images with structured annotations for document understanding tasks
[CORD Dataset](#)
- 4. Synthetic Invoice Dataset:
 - a. Source: AI-generated invoice PDFs and forms
 - b. Content: Synthetic invoices with varied layouts, fonts, OCR noise, and formatting inconsistencies

Methodology:

1. Document Ingestion Pipeline:
 - a. Convert PDF invoices and forms into images using pdf2image
 - b. Apply preprocessing techniques such as deskewing, contrast enhancement, and noise reduction
 - c. Extract baseline OCR text using Tesseract OCR
2. Multi-Modal Extraction Agent:
 - a. Pass both document images and OCR text to a vision-language model
 - b. Extract structured invoice fields using prompt-based extraction templates
 - c. Generate Pydantic-based structured outputs for extracted data
3. Layout-Aware Processing:
 - a. Handle multi-page and varied-layout documents
 - b. Maintain page-level contextual understanding during extraction
4. Validation Agent:
 - a. Validate extracted totals, dates, invoice formats, IBAN/SWIFT fields, and vendor information
 - b. Perform semantic validation using ChromaDB-indexed vendor records
 - c. Detect discrepancies using confidence scoring mechanisms
5. Auto-Correction Agent:
 - a. Re-query low-confidence regions using focused image crops
 - b. Apply fuzzy matching using rapidfuzz for vendor normalization
 - c. Correct extraction inconsistencies automatically
6. Database Population and Audit Trail:
 - a. Store validated invoice records in SQLite or PostgreSQL
 - b. Maintain audit trail including OCR output, extracted fields, corrections, and validation logs



7. Dashboard and Reporting:
 - a. Build Gradio interface for document visualization and extraction monitoring
 - b. Display original document alongside extracted JSON and validation status
 - c. Generate monthly reconciliation and analytics reports using Pandas
8. Evaluation:
 - a. Measure field-level extraction accuracy
 - b. Evaluate validation pass rate and correction effectiveness
 - c. Analyze runtime performance and OCR quality

Challenges:

1. Diverse Document Layouts: Handle invoices and forms with varying templates, orientations, and structures
2. OCR Noise and Image Quality: Process blurred, skewed, low-resolution, and noisy scanned documents
3. Multi-Modal Understanding: Combine OCR text and visual understanding effectively for accurate extraction
4. Validation Accuracy: Detect incorrect totals, vendor mismatches, and formatting inconsistencies reliably
5. Hallucination in VLM Outputs: Prevent generation of incorrect fields or fabricated invoice values
6. Multi-Page Document Handling: Maintain contextual consistency across pages of large invoices and forms
7. Scalability: Efficiently process large batches of invoices and enterprise documents

References:

1. Tesseract OCR Documentation <https://tesseract-ocr.github.io/>
2. pdf2image Documentation <https://pypi.org/project/pdf2image/>
3. Pillow Documentation <https://pillow.readthedocs.io/en/stable/>
4. Chroma Vector Database Documentation <https://docs.trychroma.com/>
5. RapidFuzz Documentation <https://rapidfuzz.github.io/RapidFuzz/>
6. SQLite Documentation <https://www.sqlite.org/docs.html>

Capstone Project 6

Title: Agentic Supply Chain Disruption Predictor & Simulation Engine

Overview and Problem Statement:

This project aims to develop an AI-powered supply chain disruption prediction and simulation system that proactively monitors global risk signals, predicts potential supply chain disruptions, forecasts demand impact, and simulates mitigation scenarios using multi-agent AI workflows, Retrieval-Augmented Generation (RAG), time-series forecasting, and discrete-event simulation techniques.

Modern supply chains are highly interconnected networks involving suppliers, manufacturers, logistics providers, ports, warehouses, and retailers across multiple countries. Disruptions such as extreme weather events, geopolitical conflicts, labor strikes, port closures, tariffs, or factory shutdowns can significantly affect inventory availability, delivery timelines, and revenue.

Most organizations react only after disruptions occur because monitoring large-scale real-time data sources manually is difficult and inefficient. Businesses require intelligent systems capable of continuously analyzing news feeds, weather alerts, shipping indices, and logistics signals to identify risks before they escalate.

The proposed system will build a LangGraph-orchestrated multi-agent platform that continuously ingests disruption-related information, classifies supply chain risks, forecasts demand fluctuations, simulates disruption scenarios, and generates mitigation recommendations. The platform aims to support proactive risk management, improve operational resilience, and assist organizations in strategic supply chain planning.

Dataset:

The project will utilize a combination of public datasets, APIs, and synthetic disruption datasets:

1. SupplyChainNet Dataset:
 - a. Source: Kaggle Supply Chain datasets
 - b. Content: Historical supply chain transactions, shipping records, supplier information, logistics delays, and disruption events
[Supply Chain Dataset \(Kaggle\)](#)
2. Freightos Baltic Index Data:
 - a. Source: Public Freightos shipping index



- b. Content: Container shipping rate indices, freight trends, and logistics cost fluctuations
[Freightos Baltic Index](#)
- 3. Open-Meteo Weather API:
 - a. Source: Open-Meteo API
 - b. Content: Historical and forecast weather data for logistics hubs and shipping regions
[Open-Meteo API](#)
- 4. News and RSS Feed Data:
 - a. Source: Reuters, Bloomberg, Supply Chain Dive, and public RSS feeds
 - b. Content: News articles related to labor strikes, geopolitical risks, tariffs, factory shutdowns, and logistics disruptions
[Reuters Business News RSS](#), [Google News API](#)
- 5. Synthetic Supply Chain Events:
 - a. Source: AI-generated synthetic disruption events
 - b. Content: Simulated demand shocks, disruption narratives, supplier failures, and logistics incidents

Methodology:

- 1. Real-Time Data Ingestion Agent:
 - a. Continuously collect data from RSS feeds, logistics APIs, weather APIs, and shipping indices
 - b. Extract disruption-related events using feedparser and NLP pipelines
 - c. Store incoming disruption signals in structured format
- 2. News and Event Analysis Agent:
 - a. Analyze news articles and logistics alerts using LLMs
 - b. Identify disruption categories such as weather, geopolitical, logistics, raw material shortages, and demand shocks
- 3. Weather Risk Monitoring Agent:
 - a. Fetch weather forecasts using Open-Meteo API
 - b. Detect extreme weather events affecting ports, transportation hubs, and manufacturing facilities
- 4. Risk Classification Agent:
 - a. Build LangGraph-based orchestration workflow
 - b. Use DistilBERT-based classifiers to categorize supply chain risks
 - c. Generate supplier-level and trade-lane risk scores
- 5. Demand Forecasting Agent:
 - a. Train forecasting models using Facebook Prophet
 - b. Incorporate disruption risk signals into demand forecasting pipelines



- c. Predict demand deviations and inventory fluctuations
- 6. Simulation Agent:
 - a. Simulate supply chain disruptions using SimPy discrete-event simulation
 - b. Model suppliers, warehouses, ports, and retailers as network nodes
 - c. Run Monte Carlo simulations to estimate stockout probability and revenue impact
- 7. Mitigation Recommendation Agent:
 - a. Generate natural-language mitigation recommendations using LLMs
 - b. Suggest alternate suppliers, route changes, safety stock adjustments, and inventory actions
- 8. Dashboard and Alerting:
 - a. Build Gradio dashboard for risk visualization and scenario analysis
 - b. Display disruption heatmaps, timelines, and simulation outcomes
 - c. Trigger alerts for high-risk scenarios
- 9. Evaluation:
 - a. Measure risk classification accuracy
 - b. Evaluate demand forecast deviation
 - c. Assess simulation realism and mitigation recommendation quality

Challenges:

- 1. Real-Time Data Integration: Continuously process multiple external data sources reliably
- 2. Risk Signal Extraction: Identify meaningful disruption indicators from noisy news and weather data
- 3. Forecasting Uncertainty: Handle uncertainty in demand forecasting under disruption conditions
- 4. Simulation Complexity: Model realistic supply chain behavior and interconnected dependencies
- 5. Multi-Agent Coordination: Synchronize ingestion, forecasting, simulation, and mitigation agents effectively
- 6. Scalability: Efficiently manage large-scale supplier networks and logistics data
- 7. Recommendation Reliability: Generate actionable and business-relevant mitigation strategies

References:

- 1. Facebook Prophet Documentation <https://facebook.github.io/prophet/>
- 2. SimPy Documentation <https://simpy.readthedocs.io/en/latest/>
- 3. Feedparser Documentation <https://feedparser.readthedocs.io/en/latest/>
- 4. DistilBERT Model Documentation <https://huggingface.co/distilbert-base-uncased>



Capstone Project 7

Title: Enhancing Customer Service with Agent-powered Chatbots

Overview and Problem Statement:

This project aims to develop an intelligent multi-agent customer service chatbot system for e-commerce platforms using Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and agentic AI workflows. The system will provide fast, personalized, and context-aware customer support across multiple customer touchpoints such as chat interfaces, customer support portals, and web applications.

E-commerce platforms often struggle to handle large volumes of customer queries efficiently. Traditional chatbots frequently fail to provide accurate, contextual, and personalized responses, resulting in poor customer satisfaction, delayed issue resolution, and increased support costs. Businesses also face challenges in managing complex customer scenarios such as order tracking, refunds, returns, product recommendations, and escalation handling.

The proposed system will leverage multi-agent architecture, intelligent reasoning strategies, and dynamic knowledge retrieval to automate customer support workflows and improve overall customer experience.

Dataset:

The project will utilize a combination of publicly available and synthetic datasets:

1. Customer Service Conversations:
 - Source: Kaggle's [Customer Support on Twitter](#) dataset
 - Content: Real customer service interactions from various companies
2. E-commerce Product Catalog:
 - Source: [Amazon product dataset](#) (a subset focused on electronics and home appliances)
 - Content: Product descriptions, specifications, prices, and customer reviews
3. Synthetic E-commerce Queries:
 - Source: Generated using GPT-5 or similar language models
 - Content: Simulated customer queries specific to e-commerce scenarios

Methodology:

1. Data Preparation:

- a. Collect and preprocess customer support conversations and product catalog datasets
 - b. Clean and organize product and customer interaction data
 - c. Generate synthetic customer queries for training and testing scenarios
2. Agentic Architecture Development:
 - a. Design multi-agent workflows for customer query handling
 - b. Create specialized agents for order tracking, returns, escalation handling, and product recommendations
 - c. Implement task decomposition and reasoning strategies
3. RAG Pipeline Integration:
 - a. Build vector database for product and support knowledge retrieval
 - b. Retrieve relevant contextual information during customer interactions
 - c. Integrate semantic search for accurate response generation
4. AI Chatbot Development:
 - a. Develop conversational chatbot using LLMs
 - b. Maintain customer conversation history and context awareness
 - c. Generate personalized and context-aware responses
5. Integration and Automation:
 - a. Build web-based chatbot interface using Streamlit or Gradio
 - b. Integrate APIs for order lookup and customer support workflows
 - c. Automate support workflows and escalation pipelines
6. Evaluation:
 - a. Measure chatbot performance using response accuracy, latency, and customer satisfaction metrics
 - b. Evaluate agent coordination and recommendation quality

Challenges:

1. Task Decomposition: Break complex customer queries into manageable subtasks for specialized agents
2. Multi-Agent Coordination: Ensure efficient collaboration between multiple AI agents
3. Context Retention: Maintain customer conversation history and contextual understanding during interactions
4. Product Knowledge Retrieval: Efficiently manage and retrieve product information using RAG pipelines
5. Scalability: Support high-volume customer interactions with low response latency
6. Personalization: Generate personalized recommendations and responses based on customer history
7. Hallucination and Accuracy: Prevent inaccurate or misleading AI-generated responses



References:

1. Lewis, Patrick, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." arXiv preprint arXiv:2005.11401.
2. Henderson, Matthew, et al. 2019. "A repository of conversational datasets." arXiv preprint arXiv:1904.06472.
3. Serban, Iulian Vlad, et al. 2017. "A deep reinforcement learning chatbot." arXiv preprint arXiv:1709.02349.

Capstone Project 8

Domain: Financial Services

Techniques: NLP, Machine Learning, Large Language Model (LLM)

Title: Personalized Financial Advisor using Large Language Model (LLM)

Overview and Problem Statement:

The field of finance can be complex and overwhelming for individuals seeking personalized financial advice. In order to make informed decisions regarding investments, retirement planning, budgeting, and financial products, individuals often require guidance from financial experts. The aim of this project is to develop an Intelligent Financial Advisor powered by a Large Language Model (LLM) to provide personalized financial advice and guidance to individuals. By leveraging NLP and machine learning techniques, the Intelligent Financial Advisor will assist users in making informed financial decisions and achieving their financial goals.

Specific Challenges:

1. Training and fine-tuning a Large Language Model (LLM) on financial datasets
2. Ensuring accurate understanding and response generation for financial queries.

Data Description:

Dataset: Alpha Vantage Financial Market Data Source: Alpha Vantage (<https://www.alphavantage.co/>). Alpha Vantage is a provider of financial market data APIs that offer real-time and historical data for various financial instruments such as stock prices, technical indicators, sector performance, exchange rates, and more. This data should be utilized to train and fine-tune the Large Language Model (LLM). Real-time financial data and market news and trends should be incorporated to provide current and accurate financial advice.

Applications:

1. Improved financial decision-making and goal achievement for individuals.
2. Offering personalized recommendations for investment portfolios, retirement plans, insurance policies, and other financial products based on individual goals and risk tolerance.

References:

1. Yang, Liu and Wang. 2023. FinGPT: Open-Source Financial Large Language Models. *ArXiv, abs/2306.06031*.



Capstone Project 9

Domain: Human Resources and Technology

Techniques: Large Language Model (LLM) Fine-Tuning, Natural Language Processing (NLP), Text-to-Speech, Video Generation

Title: Automated Video Creation from Resumes Using GPT

Overview and Problem Statement:

Creating a concise and engaging video summary of a resume can significantly enhance the recruitment process. Traditional methods of resume screening are time-consuming and may overlook key aspects of a candidate's profile. This project addresses the need for an automated solution to convert text-based resumes into 1-minute video summaries, effectively showcasing the candidate's skills, experiences, and achievements. The project involves fine-tuning a GPT model to generate a script from a resume and utilizing text-to-speech and video generation technologies to create the video.

Dataset:

The primary dataset used for this project will be the [Resume Dataset](#) from Kaggle. This dataset contains a diverse collection of resumes, covering various industries and roles. The resumes are annotated to highlight key sections such as contact information, summary, skills, work experience, education, and achievements. This comprehensive dataset will be instrumental in training the model to accurately interpret and summarize resume content.

Specific Challenges:

- Ensuring the model accurately interprets the context and importance of various resume sections to create a coherent and relevant script.
- Tailoring the video script to reflect the unique strengths and experiences of each candidate.
- Generating human-like and engaging scripts that effectively summarize the resume content.
- Integrating text-to-speech technology to produce natural-sounding narration and combining it with relevant visual elements to create a professional video.

References:

1. Text-to-Video Generative AI Models: The Definitive List:
<https://aibusiness.com/nlp/ai-video-generation-the-supreme-list>
2. Everything to Know About OpenAI's New Text-to-Video Generator, Sora:
<https://www.scientificamerican.com/article/sora-openai-text-video-generator/>
3. Resume Video Maker: <https://www.steve.ai/resume-video-maker>

Capstone Project 10

Domain: R & D

Techniques: Large Language Model (LLM) Fine Tuning, Computer Vision, NLP

Title: Gemini-SCICAP: Enhancing Scientific Figure Captioning with a Language Model

Overview and Problem Statement:

Scientific figures often contain crucial information, and providing accurate captions is essential for better comprehension. Existing generic captioning models may not capture the specialized terminology and context found in scientific literature. This project addresses the need for a dedicated model for scientific image captioning. This project involves fine tuning the Gemini Large Language Model (LLM) to generate accurate and contextually relevant captions for scientific figures. A model capable of understanding and describing complex scientific visuals will be created, combining the power of NLP with computer vision

Dataset

SCICAP (Scientific Captioning) is a large-scale image captioning dataset containing real-world scientific figures and captions. The dataset is constructed using over two million images from more than 290,000 papers collected and released by arXiv. It covers a wide range of scientific domains, making it a comprehensive resource for training and evaluating the model.

Specific Challenges

- **Scientific Terminology:** Adapting the language model to understand and generate captions with domain-specific scientific terminology.
- **Complex Visuals:** Handling intricate scientific visuals that may include charts, graphs, and diagrams.
- **Contextual Understanding:** Ensuring the model captures the context of the scientific content to provide informative and coherent captions.
- **Multi-Modal Learning:** Integrating both text and visual information for effective image captioning.

References

1. Gemini: A Family of Highly Capable Multimodal Models
2. Gemini: An Overview of Multimodal Use Cases
3. SCICAP Dataset: A Large-Scale Scientific Image Captioning Benchmark

Capstone Project 11

Domain: Legal Studies

Techniques: Natural Language Processing, Generative AI

Title: AI Patent Advisor: Leveraging Large Language Models for Patent Analysis and Technology Transfer Facilitation

Overview:

With the exponential growth of patent databases, extracting valuable insights and facilitating technology transfer has become increasingly challenging. This project aims to develop an AI-powered advisor that can analyze patents, provide comprehensive summaries, and recommend potential commercial applications and licensing opportunities. By fine-tuning the model on the AI-Growth-Lab patents and claims dataset, sourced from various patent repositories, the AI Patent Advisor will empower inventors, businesses, and legal professionals to navigate the complex landscape of patent law and innovation.

Dataset:

The AI-Growth-Lab patents and claims dataset (https://huggingface.co/datasets/AI-Growth-Lab/patents_claims_1.5m_train_test) will serve as the foundation for training the AI Patent Advisor. This dataset comprises a vast collection of patents and their associated claims, covering diverse technical fields and industries. With over 1.5 million patent documents, the dataset provides a rich source of information for training and fine-tuning the LLM model. Each patent document includes detailed descriptions, claims, and metadata, enabling comprehensive analysis and understanding of patented technologies.

Methodology:

- Data Collection and Preprocessing: Gather AI-Growth-Lab patents and claims, preprocess to clean noise, standardize formatting, and tokenize text
- Fine-tuning LLM: Utilize state-of-the-art LLM architecture, fine-tune on AI-Growth-Lab dataset for patent-specific language adaptation.
- Analyze patents, extract key concepts, and generate concise summaries for efficient knowledge extraction.
- (Optional) Implement semantic matching algorithms to map patented inventions to potential commercial applications and licensing opportunities.
- Design a user-friendly interface enabling users to input patents, explore summaries, and receive technology transfer recommendations.



- (Optional) Testing and Validation: Assess AI Patent Advisor's performance, including accuracy of summaries, relevance of transfer recommendations, and overall usability.
- (Optional) Deployment and Maintenance: Deploy the AI Patent Advisor, ensuring scalability and reliability, with protocols for regular maintenance and updates to align with evolving patent language and industry trends.

Challenges:

- Handling the complexity and variability of patent language and terminology; Ensuring the accuracy and relevance of patent summaries and technology transfer recommendations

Significance:

The AI Patent Advisor project holds immense potential to transform the landscape of patent analysis and technology transfer facilitation: It will enable inventors, businesses, and legal professionals to efficiently analyze patent documents and extract valuable insights; bridge the gap between patented inventions and commercial applications by identifying potential licensing opportunities and strategic partnerships Overall it will promoter innovation by accelerating the pace of innovation by facilitating knowledge dissemination and collaboration within the patent ecosystem.



Capstone Project 12

Title: A Versatile LLM-Based Code Generation Platform for Varied Complexity Tasks

Overview and Problem Statement:

This project aims to develop an advanced code generation system powered by Large Language Models (LLMs) that can interpret various input types and generate appropriate code solutions for tasks of varying complexity. The system will be designed to understand the nuances of different programming tasks, break them down into manageable components, and provide a comprehensive solution that includes flow charts, task lists, test cases, and functional code.

Methodology:

1. Data Preparation: Create multi-modal dataset (text, audio, images)
2. LLM Setup: Select and fine-tune LLM for code generation
3. Input Processing: Develop multi-modal input handling (text, audio, image)
4. Complexity Analysis: Design task complexity assessment algorithm
5. Clarification System: Implement LLM-driven question generation
6. Task Visualization: Create flow chart and task list generator
7. Code Generation: Develop core functionality for multiple languages
8. Testing Module: Automate test case generation and execution
9. Refinement Process: Design feedback loop for code improvement
10. User Interface: Create adaptable, user-friendly interface
11. Evaluation: Conduct extensive testing and user studies

Challenges:

1. Input Processing: Handle diverse input types accurately
2. Complexity Analysis: Assess task complexity reliably
3. Context Comprehension: Ensure LLM understands complex programming tasks
4. Code Quality: Generate high-quality, industry-standard code
5. Scalability: Design for wide range of tasks and languages
6. Ethics: Address code ownership and responsible AI use
7. User Experience: Create intuitive interface for varied expertise levels

References:

1. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.



2. Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., ... & Sifre, L. (2022). Competition-level code generation with AlphaCode. *Science*, 378(6624), 1092-1097.
3. Jiang, N., Shen, Y., Chen, Z., Yang, C., Zhang, L., & Cao, Y. (2023). InstructCoder: A generative model for code infilling and rewriting based on fine-grained code instructions. *arXiv preprint arXiv:2305.11766*.



Capstone Project 13

Title: Interactive Root Cause Analysis in IT Operations Using Generative AI: A Case Study with OpenStack Infrastructure Logs

Overview and Problem Statement:

This project aims to develop an interactive root cause analysis (RCA) system using Generative AI to understand and explain incidents in IT departments. RCA is a critical task for identifying the underlying causes of incidents, system failures, and performance bottlenecks. Traditional RCA is a time-consuming process often dependent on expert input.

Generative AI can automate and enhance this process by analyzing system logs and performance data, detecting patterns, and explaining potential root causes interactively. A key aspect of this project is to make the AI model amenable to queries, allowing IT teams to ask the AI for detailed insights about incidents, such as:

- "What caused the server outage?"
- "Explain the root cause of the storage failure."
- "What factors contributed to this anomaly?"

The project will explore how transformer-based models can be used for interactive troubleshooting in a cloud infrastructure environment, focusing on the OpenStack ecosystem.

Dataset:

1. OpenStack Infrastructure Logs (LogHub):

- **Description:** The dataset includes logs from OpenStack cloud infrastructure, covering key services such as Nova (compute), Neutron (networking), and Cinder (storage). These logs capture events, system activities, and error messages, providing a rich dataset for RCA in cloud environments. This dataset can be used to generate and explain potential root causes for service failures, anomalies, and performance issues in OpenStack infrastructure.
- **Source:** [OpenStack Logs on LogHub](#)

Methodology:

Data Preprocessing:

- Clean and normalize OpenStack logs.
- Use NLP to structure log data.
- Apply anomaly detection for incidents.



Model Selection:

- Use transformers for interactive root cause generation.
- Implement LSTM/autoencoders/logBERT for anomaly detection.
- Build an interactive UI for querying the AI.

Training & Validation:

- Train on OpenStack logs for accurate root cause insights.
- Validate with real and synthetic incidents.
- Test query responses for clarity.

Automation & Interaction:

1. Automate real-time log analysis.
2. Enable AI queries for root cause insights.
3. Integrate with ITSM platforms for incident management.

Challenges:

Data Complexity and Noise:

- OpenStack logs are complex and can be noisy, with incomplete data or inconsistencies across different services (e.g., compute, networking). Effective log parsing and filtering are crucial for ensuring meaningful RCA.

Interactive Explainability:

- One of the main challenges is ensuring that the AI can explain root causes in a human-understandable way. The model needs to provide not only predictions but also justifications, ensuring the results are interpretable and actionable.

References:

1. LogHub - OpenStack Dataset: <https://github.com/logpai/loghub>.
2. LogBERT: Log Anomaly Detection via BERT - <https://arxiv.org/abs/2103.04475>

Capstone Project 14

Title: AI-Powered Plant Disease Detection and Farmer Assistance Using Generative AI

Overview and Problem Statement:

This project focuses on helping farmers identify plant diseases through image recognition and providing actionable advice. Farmers will upload images of their crops, and the system will use the MobileNetV2 model or a Vision Language model to detect diseases and Generative AI to generate clear, helpful explanations.

The primary objective is to build a system that provides accurate disease detection and recommendations. An optional challenge is to enhance the system with multilingual support, offering explanations in regional languages (e.g., Hindi, Tamil, Telugu), but this is not mandatory.

Dataset:

1. MobileNetV2 Plant Disease Dataset (Hugging Face):
 - Pretrained MobileNetV2 model trained on a large collection of plant disease images.
2. Agricultural Dataset:
 - Agricultural terms, disease descriptions, and treatment recommendations for training the Generative AI model to provide disease explanations and treatment advice.

Methodology:

1. Data Preprocessing:
 - Preprocess plant images for uniformity (resizing, noise removal).
 - Prepare agricultural phrases and disease explanations for the model.
2. Model Selection:
 - MobileNetV2 for Plant Disease Detection:
 - Use the pretrained MobileNetV2 model for identifying diseases from plant images.
 - Generative AI (LLMs) for Disease Explanations:
 - Use an LLM to generate disease explanations and treatment suggestions, tailored to the detected disease.
 - Optional Multilingual Support (Challenge):
 - Participants may choose to integrate multilingual capabilities, translating disease explanations into regional languages, but this remains optional.

3. Training & Validation:

- Train the models to deliver accurate disease identification and explanations.
- Validate the system using real-world plant disease images and ensure clarity of explanations.

4. User Interaction & Query Support:

- Build a user-friendly interface where farmers can upload images and receive instant disease diagnoses and treatment suggestions.
- Allow users to ask additional queries, like “How to treat this disease?” and get detailed responses.

Challenges:

1. Image Quality:

- Ensure the system handles varying image quality effectively (e.g., blurred or low-light images).

2. Optional Multilingual Support:

- Accurately translating agricultural terms and maintaining the clarity of explanations across different languages.

3. Digital Literacy:

- Create an intuitive and simple user interface that accommodates varying levels of digital literacy among farmers.

References:

1. MobileNetV2 for Plant Disease Detection (Hugging Face): Available at [MobileNetV2 Plant Disease Model](#).
2. Optional Translation Models (IndicTrans, mBART): Indic NLP Project. Available at [IndicTrans](#).



Capstone Project: 15

Domain: Customer Service and Conversational AI

Techniques: Natural Language Processing (NLP), Large Language Models (LLMs), Sentiment Analysis, Intent Recognition, Topic Modeling

Title: Customer Conversational Intelligence Platform Powered by an LLM Agent

Overview and Problem Statement:

This project aims to develop a state-of-the-art Customer Conversational Intelligence Platform powered by a Large Language Model (LLM) agent. The LLM's advanced language understanding will drive the analysis of customer interactions across diverse channels (chatbots, call centers, email, social media). The platform will extract actionable insights from this data, enabling businesses to optimize customer service processes and significantly enhance the overall customer experience.

Datasets:

Name: Relational Strategies in Customer Interactions (RSiCS)

- Description: This dataset contains a corpus for improving the quality and relational abilities of Intelligent Virtual Agents.
- Link: [Link to the dataset](#)

Name: 3K Conversations Dataset for ChatBot from Kaggle

- Description: The dataset includes various types of conversations such as casual or formal discussions, interviews, customer service interactions, and social media conversations.
- Link: [Link to the dataset](#)

Name: Customer Support on Twitter Dataset from Kaggle

- Description: This is a large corpus of tweets and replies that can aid in natural language understanding and conversational models.
- Link: [Link to the dataset](#)

Challenges:

1. Data Collection: Gather customer conversations from diverse sources like voice calls, chat transcripts, emails, and social media interactions.
2. Use LLM-Agent for:
 - a. Sentiment Analysis – accurate detection of customer emotions (positive, negative, neutral) and granular sentiment categories (frustration, satisfaction, inquiry, etc.) throughout conversations.



- b. Intent Recognition - understanding the underlying purpose behind customers' queries, enabling tailored responses and resolutions
- c. Topic Modeling - discovering recurring themes and patterns within conversations, highlighting trending issues, feedback topics, and potential areas for improvement.
- d. Agent Performance Evaluation - Analyzing agent interactions to provide constructive feedback, identifying training needs, and recognizing exceptional service.
- e. LLM-Driven Real-time Recommendations - Empowering agents with suggestions for next-best actions or responses during active conversations, optimizing outcomes.

Methodology:

Select GPT2/GPT3, fine-tune the LLM agent extensively on a large dataset of customer conversations annotated for sentiment, intent, topics, etc. Develop ML algorithms to support the LLM agent. The primary focus will be on the LLM's ability to perform sentiment analysis, intent recognition, topic modeling, and agent performance assessment. Utilize platforms like SageMaker or equivalent to automate the ML workflow.

Example:

Customer: Hello, I ordered a laptop from your website, and it's been a week, but I haven't received it yet. Can you help me track my order?

Platform Analysis:

Categorization: Inquiry about order tracking.

Sentiment Analysis: Neutral sentiment.

Resolution Status: Unresolved.

Support Agent: Hi there! I apologize for the delay in your order. Could you please provide me with your order number? I'll check the status for you.

Customer: My order number is 123456789.

Platform Analysis:

Categorization: Providing order information.

Sentiment Analysis: Neutral sentiment.

Resolution Status: In progress.



Support Agent: Thank you for providing the order number. Let me check that for you.
[Platform sends a real-time request to the order tracking system]

Platform Analysis:

Real-time Analysis: The platform receives updated order tracking information. The laptop is currently in transit and is expected to arrive in two days.

Support Agent: Good news! Your laptop is on its way and should be delivered within the next two days. Here's your tracking number: ABC123XYZ. You can use this number to monitor its progress.

Customer: Thank you for the information. I appreciate your help.

Platform Analysis:

Sentiment Analysis: Positive sentiment.

Resolution Status: Resolved.

Support Agent: You're welcome! If you have any more questions or need further assistance, feel free to ask. Have a great day!

Significance:

As companies accumulate immense volumes of customer interaction data, the ability to unlock meaningful insights and streamline customer service processes becomes a competitive advantage. The envisioned platform, with its real-time analysis capabilities, has the potential to revolutionize customer service, ultimately translating into greater customer satisfaction, increased operational efficiency, and a strengthened market position for businesses.

Reference:

1. Conversational Health Agents: A Personalized LLM-Powered Agent Framework, Mahyar Abbasian, Iman Azimi, Amir M. Rahmani, Ramesh Jain:
<https://arxiv.org/html/2310.02374v4>
2. Building a Conversational AI Agent with Long-Term Memory Using LangChain and Milvus, Zilliz:
https://medium.com/@zilliz_learn/building-a-conversational-ai-agent-with-long-term-memory-using-langchain-and-milvus-0c4120ad7426